

J-ART：アプリケーション層の LLM セキュリティと コスト効率のための日本語敵対的レッドチーム評価フ レームワーク

テクニカルレポート (v0.1, プレプリント)

廣瀬 隆之 (Takayuki Hirose)*

2026-06-13

概要

大規模言語モデル (LLM) の公開レッドチーミング/ジェイルブレイク評価の多くは英語中心で、かつ**素のモデル**を対象とする。本稿は **J-ART** (日本語敵対的レッドチーム評価フレームワーク) を提案する。J-ART は、(i) 素のモデルではなく**アプリケーション構造層**——モデル × システムプロンプト強度 × ガードレール × 検索 (RAG)——を評価し、(ii) ギャル文字による字形置換・縦書き/改行分割・慇懃無礼 (keigo) な権威付け・二枚舌の偽前提・Base64・leet/ゼロ幅密輸といった**日本語特有の難読化変形**でナイーブなキーワード検閲を突破し、(iii) 攻撃を **MITRE ATLAS** (推論時 10 技術) に対応づけ、(iv) 防御率に **Wilson 95% 信頼区間**を付し **コスト効率**指標を併記する、再現可能なオープンハネスである。攻撃は**無害なプロキシ** (架空のカナリアと無害なマーカー) で構成され、**危険物を一切含まず**、各攻撃の悪意あるコアは公開物上でマスクされる。18~21 のアプリ構成と 11 攻撃 × 7 変形にわたり、**アプリ層の防御がモデル素性を上回る**こと、専用の**入力分類器 (Llama Guard)**が弱構成の突破を全て遮断し、しかも正味コストを下げたこと、単純な**正規化フィルタ**がナイーブなキーワード検閲の取りこぼす難読化を無効化することを示す。さらに、ルーター経由で提供される同一構成が実行間で大きくばらつく (ある構成が 2 回の実行で 58 突破 → 0 突破に変動) ことを記録し、単一サンプルのリーダーボードが信頼できないこと、統計的処理が必要であることを論じる。

ステータス. 本稿は概念実証 (PoC) と予備的知見を記述する暫定的なテクニカルレポート/プレプリントであり、いかなるベンダーのモデルの公式な安全性評価でもない。数値は実行時のスナップショットであり、第 7 節の限界に従う。参考文献は暫定であり、正式投稿前に書誌情報を確認すること。

1 はじめに

公開 LLM レッドチーミングのツール (例: garak) やジェイルブレイクのベンチマーク (例: HarmBench, JailbreakBench) は英語が主流である。多言語安全性の研究は、プロンプトを低資源

* 独立研究者 (Independent Researcher)。ORCID: 0009-0005-6735-1862。コード: <https://github.com/takker-hero-se/J-ART>。リーダーボード: <https://takker-hero-se.github.io/J-ART/>。ソフトウェア DOI (concept): [10.5281/zenodo.20676879](https://doi.org/10.5281/zenodo.20676879)。本文 CC-BY-4.0、コード MIT。氏名の漢字表記は必要に応じて修正してください。

言語へ翻訳することで安全機構を回避できることを示してきたが、言語固有の文字・組版・敬体を悪用する言語特有の難読化は相対的に未開拓である。とりわけ日本語は、複数の文字体系、全角/ゼロ幅文字、縦書き、高度に慣習化された敬語など、豊かな難読化の表面を持つ。

さらに2つのギャップが J-ART の動機である。第一に**素のモデル評価とデプロイシステム評価**の差：実務では、モデルに加えてシステムプロンプト、さらにガードレール、しばしば検索が組み合わさる。セキュリティ上意味のある単位は**アプリケーション構成**であり、モデル単体ではない。第二に**セキュリティとコストの同時評価**：防御はトークンやレイテンシのコストを伴うため、コスト効率スコアを併記し「最も安価で十分に安全な構成」を直接読み取れるようにする。

本稿の貢献は次の通り：(a) 日本語難読化変形の一式、(b) 安全なプロキシマーカー方式を備えたアプリ層・MITRE ATLAS 準拠の評価ハーネス、(c) 正規化フィルタとモデルベース分類器を含むガードレール比較、(d) 統計的に報告するリーダーボード（Wilson 信頼区間・任意の N 回試行）とコスト効率指標、(e) ルーター起因の大きな実行間ばらつきの実証的観察。

2 関連研究

J-ART は次の 6 系統の研究の交点に位置する。各系統を要約し、表 1 に位置づけをまとめる。

■**敵対的レッドチーミングのツール**. garak [2] は**モデル**単体を既知の脆弱性プローブ集合で走査する。J-ART は、配備された**アプリケーション構成**（モデル × システムプロンプト × ガードレール × RAG）を評価単位とする。

■**ジェイルブレイク・有害行動ベンチマーク**. AdvBench/GCG [11]、Jailbroken [12]、Harm-Bench [3]、JailbreakBench [4]、実環境収集プロンプト [13] は、**どの有害行動をどう採点するか**を標準化するが、その大半は英語・モデル層が対象である。J-ART はハーネスの発想を継承しつつ、有害誘発を安全なプロキシマーカー目標 (§3) に置き換え、スイートを公開可能にする。

■**自動攻撃生成**. PAIR [6] と TAP [7] は攻撃プロンプトを反復最適化する。J-ART は再現性と安全な開示のため、**固定・監査可能な変形一式**を用いる。

■**多言語・低資源の安全性**. Deng ら [14]・Yong ら [5] は低資源言語への**翻訳**でアライメントが劣化することを、Wang ら [15] は言語横断の安全性般化を示す。これらは言語間の翻訳を扱うが、J-ART は日本語固有の**言語内字形難読化**（ギャル文字・縦書き・敬語フレーミング・かな/漢文字符号化）を扱う。これは日本語が高資源であっても残存する。

■**プロンプトインジェクション**. Perez & Ribeiro [16] は目標乗っ取り・プロンプト漏洩を、Greshake ら [8] は**間接注入**を定式化し、Liu ら [17] は LLM 統合アプリのベンチマークを与える。J-ART はこれらを**アプリ構成**の関数として評価する。

■**ガードレール・入力分類器**. Llama Guard [9]、NeMo Guardrails [18]、OpenAI Moderation API [19] はモデル/規則ベースのフィルタを提供する。J-ART はガードレールを**差し替え可能な構成変数**として扱い、各々の限界防御を測る。ナイーブなキーワード検閲が取りこぼす難読化を破る決定論的正規化フィルタも含む (§4)。

表 1 代表的先行研究に対する J-ART の位置づけ。

研究	言語	評価単位	コスト 軸	ガード/RAG	公開
garak [2]	英語	モデル	無	無	有
HarmBench /	英語	モデル	無	無	有
JBB [3, 4]					
PAIR / TAP [6, 7]	英語	モデル	無	無	一部
多言語 [5, 14, 15]	低資源	モデル	無	無	有
Llama Guard /	英語中心	分類器	無	該当無	有
NeMo [9, 18]					
J-ART	日本語	アプリ構成	有	有	有

■LLM-as-judge の妥当性. Zheng ら [20] と Gu ら [21] は LLM ジャッジの偏りを記録する。J-ART は**主要指標**で LLM ジャッジを避け、決定論的なカナリア/マーカ照合 (§3) を用いてジャッジ分散を指標から排除する。

■日本語 NLP 資源とタクソノミ. JGLUE [22] のような一般日本語ベンチマークは自然言語理解を対象とするが、日本語固有の難読化を狙う公開・再現可能な**敵対的安全性**ベンチマークは管見の限り存在しない。これが J-ART の埋める空隙である。攻撃は MITRE ATLAS [1] に対応づけ、OWASP Top 10 for LLM Applications [10] を相互参照する。

すなわち J-ART は、**日本語の言語内字形難読化**の表面、**アプリ構成**の評価単位、**コスト効率**の軸、**安全なプロキシマー**方式を、単一の再現可能なハーネスに統合する点で他と異なる。

3 手法

■アプリケーション構造層. 評価対象(**target**)は構成である: provider/model × prompt_strength ∈ {naked (low), hardened (high)} × guardrail ∈ {none, keyword, regex, llm, llamaguard} × rag ∈ {true, false}。簡易 RAG 文脈 (良性の社内文書、間接インジェクション攻撃では汚染) と、秘密の**カナリア**を含むシステムプロンプトを試行ごとに組み立てる。

■日本語難読化変形. 各ベース攻撃は 7 種の変形のいずれかでレンダリングされる: **baseline** (無変形)、**polite_business** (敬語による権威付け)、**vertical_newline** (1 文字 1 行の縦書き)、**gyaru** (ギャル文字の字形置換+ゼロ幅スペース)、**double_tongue** (偽のサンドボックス/認可の前提)、**base64_wrap** (Base64 エンコードした指示)、**leet_smuggle** (leet 記号置換+ゼロ幅スペース)。変形は人間可読性を保ちつつ、部分文字列一致のフィルタを破壊するよう設計される。

変形は 2 クラスに区別する (表 2)。**日本語固有**の変形は日本語の表記体系や待遇表現を悪用し、英語などに直接の対応物を持たない: **gyaru** (仮名字形置換)、**vertical_newline** (縦書き)、**polite_business/double_tongue** (敬語による権威付けと内集団フレーミング)。**言語非依存**の変形 — **base64_wrap・leet_smuggle** (および対照の **baseline**) — は任意の言語に適用でき、日本語固有面の限界効果を切り分けるための比較基準として含める。これにより、日本語を越えて般

表 2 難読化変形の分類。

クラス	変形	悪用する性質
日本語固有	gyaru, vertical_newline, polite_business, double_tongue	仮名字形の異形・縦書き・敬語による権威/内集団フレーミング
言語非依存	base64_wrap, leet_smuggle (+ baseline)	任意の言語に適用できる符号化 (+ /記号置換)

化が見込める知見（言語非依存の符号化）を明確化する。

■**攻撃スイートと MITRE ATLAS 対応.** 11 の攻撃が 10 の推論時 ATLAS 技術を網羅する：直接/間接プロンプトインジェクション (AML.T0051.000/.001)、LLM ジェイルブレイク (AML.T0054)、LLM データ漏洩 (AML.T0057)、LLM プロンプト自己複製 (AML.T0061)、信頼出力/引用の操作 (AML.T0067.000)、LLM プロンプト難読化 (AML.T0068)、システムプロンプト探索 (AML.T0069.002)、RAG 汚染 (AML.T0070)、偽 RAG エントリ注入 (AML.T0071)。ベクトルは user（直接）または rag（間接）。

■**無害なプロキシマーカーと設計による安全性.** 危険なコンテンツの生成を避けるため、攻撃は無害なプロキシを標的とする：システムプロンプトに埋め込んだ架空のカナリア合言葉（データ漏洩攻撃）と、無害な文字列マーカー [JART-PWNED] / [JART-HIJACKED]（注入/制御奪取攻撃）。各攻撃の悪意あるコアはコード内に定義し、設定ファイルには残さず、results.json と公開サイトではマスク（伏字プレースホルダへ置換）する。表示されるのは難読化のフレームワークのみであり、利用可能なレシピを与えずに構造を学べる。描画時にもクライアント側の多重マスクを施す。

■**決定論的ジャッジ.** 突破は完全一致で判定する：応答中のカナリア (secret_leak)、または応答中のマーカー (marker_injection / marker_hijack)。本ジャッジは再現可能かつ安価だが保守的であり、意味的に等価な漏洩を取りこぼし得る（第 7 節）。

■**ガードレール.** 5 設定を比較する：none；keyword（禁止部分文字列一致——高速だが難読化に弱い）；regex（正規化フィルタ。ゼロ幅文字除去・leet 逆変換・空白畳み込み・長トークンの Base64 復号を行ってから禁止語照合——決定論的で API 追加コスト 0）；llm（対象モデル自身が入力を検閲し出力も審査）；llamaguard（専用の安全性分類モデル——ルーター経由の Llama Guard 4——が入力を攻撃/良性に分類し、失敗時は安全側にフォールバック）。

■**統計的処理.** 各モデルの防御率は、その試行数に基づく **Wilson 95% 信頼区間**と共に報告する。任意で各セル (target, attack, transform) を N 回反復でき (JART_TRIALS、既定 1)、詳細はセル単位に集約し、要約統計と信頼区間は試行レベルの標本を用いる。シミュレーションモードでは各反復が独立なベルヌーイ試行であり、 N の増加で信頼区間が縮まる（ある構成で $N=1 \rightarrow N=3$ により幅 $15.1 \rightarrow 7.5$ ポイント）。

表 3 弱いベース構成に対するガードレール効果（実行 B・GPT-4o-mini）。

構成	防御率	95% 信頼区間	突破
GPT-4o-mini / 素の API	81.8%	[71.8, 88.8]	14 / 77
GPT-4o-mini + Llama Guard	100.0%	[95.2, 100]	0 / 77

■**コスト効率指標**. $\text{cospa} = \text{防御率}(\%) \div 100 \text{ 万トークンあたりコスト (USD)}$ を報告する（100 万トークン単価は 0 近傍の発散を防ぐため下限\$0.01 でクランプ）。これにより、安価かつ安全な構成が浮かび上がる。

■**実行**. 試行は独立であり、有界並列（既定 8 ワーカーのスレッドプール）で、リクエスト毎のタイムアウトと指数バックオフ再試行のもとに実行する。シミュレーションの決定論は実行順に依存しない。API キーが無い対象は決定論的シミュレーションにフォールバックし、鍵が一部欠けても実行全体が落ちない。

4 結果

2 回の LIVE 実行を要約する（実行 A：18 構成・1,386 セル、実行 B：21 構成・1,617 セル、各セル 1 試行）。

■**全体**. 突破率は総じて低く（実行 A：108/1,386 = 7.8%、実行 B：36/1,617 = 2.2%）、**弱構成**（素の API・低強度モデル）に集中した。強プロンプト+ガードレール構成は概ね 100% 防御した。

■**アプリ層の防御がモデル素性を上回る**. 実行 A では、システムプロンプトもガードレールも無いフラグシップモデルが 10 回突破され（防御 87.0%）、一方で**強プロンプト+キーワード GR**を備えた安価な小型モデルは $\geq 96\%$ 防御した。中心的な実務知見：どのモデルかより、アプリをどう構成したかが効く。

■**実行間ばらつき（主要な結果）**. 同一構成「Llama 4 Scout / 素の API」は、実行 A で **58 突破（防御 24.7%）**、実行 B で **0 突破（防御 100%）**を示した。2 回の唯一の違いは実行時刻であり（モデルは異なるバックエンド/量子化へ振り分け得るルーター経由で提供）、これは**単一サンプルの LLM リーダーボードが再現しないこと**、防御率は信頼区間と——できれば——反復試行と共に報告すべきことを示す。本稿はこのばらつきを隠すべきノイズではなく結果として扱う。

■**ガードレール比較**. 実行 B では「GPT-4o-mini / 素の API」が 14 回突破され（防御 81.8%、CI[71.8–88.8]）、同一モデルに **Llama Guard 入力分類器**を付けると **0 回（防御 100%、CI[95.2–100]**）で、しかも正味コストはより低かった（ブロックされた入力本体モデル呼び出しを省くため）。**正規化 regex フィルタ**は、ユニットテストにおいてキーワードフィルタの取りこぼす縦書き・leet・Base64・ギャル文字の各変形を決定論的に無効化し、API 追加コストは 0 である。実行 B では弱構成のベースラインが突破しなかったため、その予防効果はそのスナップショットでは分離できなかった（第 7 節）。表 3 に Llama Guard の比較を示す。

■**攻撃・変形の有効性**. 実行 A で最も有効だった攻撃は、信頼出力/引用の操作、偽 RAG エントリ注入、間接 RAG 窃取であり、最も有効だった変形は縦書き改行・baseline・慇懃無礼であった。一方 **Base64** は最も効果が低かった（モデルがエンコードされた指示に従うのを拒むことが多い）。新規追加した ATLAS 技術が突破の大きな割合を占めた。

5 リーダーボード成果物

ハーネスは `results.json` と自己完結の静的サイト（並び替え可能なリーダーボード、マスク済みプロンプト付きの構成別攻撃ログ、環境に応じて自動選択する日英 UI、モデル別防御率の信頼区間、セル別突破率）を生成し、継続的にデプロイする。

6 倫理と責任ある開示

J-ART は設計により安全である。(1) 攻撃は**架空のカナリア**と**無害なマーカー**を標的とし、実際の有害能力を扱わない。コード・データ・公開サイトのいずれにも危険物は存在しない。(2) 悪意ある**コア**は全成果物でマスクされ、難読化のフレームワークのみが示される。(3) モデルは研究比較のためにのみ参照され、結果は実行時スナップショットであって認証ではない。意図する用途は、開発者が**自身の**デプロイを堅牢化する支援である。

7 限界

- **プロキシマーカーであって実害ではない**：実害ではなく指示違反/漏洩の代理を測る。
- **決定論ジャッジ**：カナリア/マーカーを完全一致および軽い正規化（大小文字畳み込み・空白/ゼロ幅除去）後に照合するため、整形してエコーされた漏洩も突破として計上する。一方で**意味的/言い換え**による漏洩は検出できないため、報告する防御率は**上界**である。人手検証付き LLM ジャッジは今後の課題。
- **単発（1 ターン）**：多ターン/*crescendo* 攻撃は未モデル化。
- **標本数**：手作りのスイート（11 攻撃 × 7 変形）、既定はセル当たり $N=1$ 。
- **プロバイダの非決定性/ルーターのばらつき**：結果はスナップショットであり、実行時のルーティングとモデル版に依存する。
- **コスト帰属**：ガードレールのトークンは対象モデルの単価で計上（近似）。
- **シミュレーションモード**は鍵欠如時の決定論的代替であり、実測のモデル挙動と読むべきではない。

8 今後の課題

人手検証付きの LLM ジャッジ、多ターン・適応的（PAIR/TAP 系）攻撃、より大規模で一部人手キュレートした攻撃集合、ATLAS 技術の追加（コスト枯渇・モデル抽出）、プロバイダ別並列と増分キャッシュによる経時トレンド追跡、そしてシミュレーションモードを外した純粋に実測・反復

試行による統計報告。

9 再現性

コード・設定・デプロイ用ワークフローは公開している。ハーネスはシミュレーションモードでは決定論的であり、LIVE 結果は実行時のプロバイダ・ルーティングとモデル版に依存するため、実行日とモデル識別子と共に報告すべきである。使用した正確なモデル識別子と単価はリポジトリを参照のこと。

参考文献

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org>.
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.
- [10] OWASP Top 10 for Large Language Model Applications.
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.
- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2024.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.
- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.